

Project #1 – Data Preprocessing

Submission Deadline: 10/11 (Fri.) 11:59 p.m.

Introduction

In the machine learning task, what come to mind first is building a model to solve different kind of tasks like regression or classification problems. In fact, machine learning tasks are not that simple. If we dive into the detail of it, we'll find out the task can be further divide into many steps, including data collection, data preparation, model selection, model training, model evaluation, parameter tuning, and prediction. Here, we specifically focus on data preparation, which is a very important part that may seriously affect the result of your models. The aim of this project is to let you design your own data preprocessing approaches. After data preprocessing are done, you can train your model and evaluate it to see if its performance is good or not.

Objectives

In this project, there're two jobs you need to do. First, you need to implement your own custom approach for data preprocessing. More specifically, you must implement the data preprocessing method in [preprocessor.py/preprocessor.hpp](#).

Second, after you've done for your implementation, your next job is to evaluate the model (Logistic regression) I provided for you in the framework and use it to predict the outcome.

Note: The second part I've already done for you, the required jobs for

you are to implement the code (data preprocessing) and evaluate the performance of the model.

(Go check [*proj1_guildline_python.pdf*](#) or [*proj1_guildline_cpp.pdf*](#) for the further detail)

Dataset Description

The data for this project are real-word data for hospitalizing patients. Each patient is described by *77 features* (F1~F77), together with a binary outcome. The project involves training and validating model with provided datasets (train_X.csv & train_y.csv) and final testing on an independent test set (test_X.csv & test_y.csv). Detailed information on patient features is available in [*Data_Description.pdf*](#).

Language:

You can implement your code by using either *Python3* or *C++*.

(just need to submit one of them)

Requirement:

1. Download the necessary library:

Python: [Numpy](#), [Pandas](#), [Scikit-learn](#) (or using environment.yml I provide in the framework).

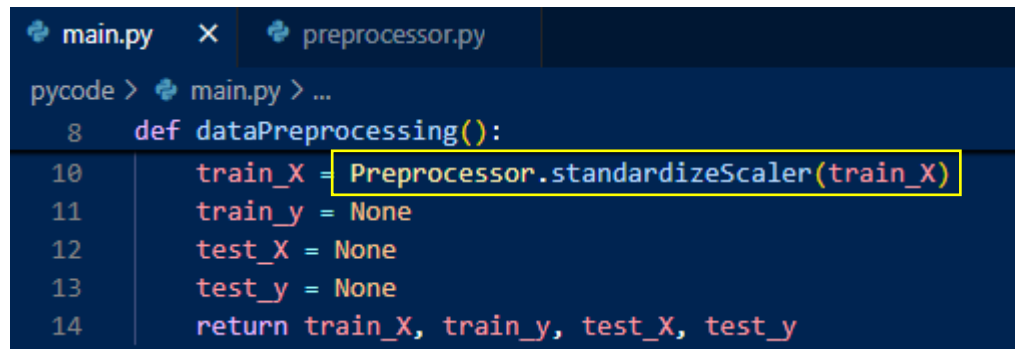
C++: [Eigen](#).

2. Preprocessing Class Implementation:

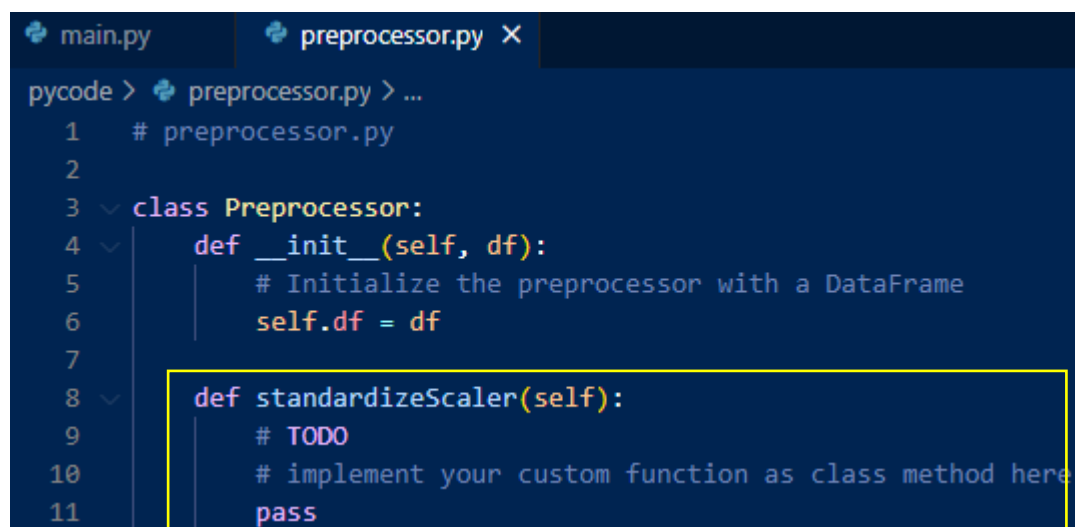
Implementing suitable preprocessing techniques such as imputation, standardization, or preprocess numeric, **customizing** these methods in **Preprocessor class** in [preprocessor.py](#)/[preprocessor.hpp](#) to fit the specific requirements of Logistic

regression model and the characteristics of the data.

e.g. If you decide to implement the standardization for your data, please implement like OOP-programming form:



```
main.py X preprocessor.py
pycode > main.py > ...
8 def dataPreprocessing():
10     train_X = Preprocessor.standardizeScaler(train_X)
11     train_y = None
12     test_X = None
13     test_y = None
14     return train_X, train_y, test_X, test_y
```



```
main.py preprocessor.py X
pycode > preprocessor.py > ...
1 # preprocessor.py
2
3 class Preprocessor:
4     def __init__(self, df):
5         # Initialize the preprocessor with a DataFrame
6         self.df = df
7
8     def standardizeScaler(self):
9         # TODO
10        # implement your custom function as class method here
11        pass
```

(Note: if you do not implement like this and we cannot understand your code, we're afraid that we will deduct some point from your project #1 in source code part)

3. Customize your own data preprocessing method:

After finishing implementation for the preprocessing methods, your need to use them to complete the data preprocessing function in main.py/main.cpp.

4. Performance measuring:

Screenshot the results of your predictions for Logistic Regression model and paste it in your report.

(Note: Do not modify the hyperparameters of the Logistic Regression model, or you'll get 0 in performance score of your project #1.)

5. Do your implementation from scratch

Do not use any machine learning libraries, except the part we've already done for you in the framework.

Grading Policy (Total 110 pt)

1. Report Spec (60%) (follow this format to write your report)

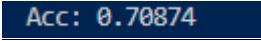
- A. Explain the concept and code detail of your **custom preprocessor class** in *preprocessor.py/preprocessor.hpp* (40%)
(Note: You need to implement the code to change values of features from word to numeric, instead of just replace it in the excel. E.g. If you find out some values are Yes or No, change it into 1 or 0)
- B. Explain the concept and code detail of your **dataPreprocessing function** in *main.py/main.cpp* (10%)
- C. Discussion (10%):
Show anything you want to discuss. Maybe some finding or the difficulty during your implementation.
- D. Bonus (10%):
 - (a) Implement MICE and explain its concept (10%)
Hint: you can refer to the methods in Logistic Regression class I provided for you, but make sure I can run your code and get your prediction of accuracy.

2. Performance (25%)

For your result of performance, you'll get:

- 25/25: if $\text{acc} \geq 0.7$
- 20/25: if $0.7 > \text{acc} \geq 0.65$
- 15/25: if $0.65 > \text{acc} \geq 0.6$
- 0/25: if $\text{acc} < 0.6$

Screenshot and **paste** your result as how the following picture looks like in your report.



3. Source code (15%)

We will check your code to see if your code is implemented like what you explain in the part (1-A) of your report. Then, **we'll run your code** by using the raw data we gave you on E3 to make sure you do not cheat in the result.

(This means that you must figure out how to convert the value of features into numeric type, or the code will throw errors if we use the data that is stored in our computer.)

If you're worried that we cannot run your code properly, you can explain how to run your code in detail in your report.

e.g. Tell us how to execute your code and where we should modify the path.

Submission format:

- You need to submit **all your code in one folder** together with your **report file**, and zip them into one **.zip** file.
- Name your code folder as “<language>_hw1_<yourStudentID>_<yourName>”,
e.g. “Cpp_hw1_312551120_陳柏憲” or “Python_hw1_312551120_陳柏憲”
- Name your report file as “report_hw1_<yourStudentID>_<yourName>.pdf”,
e.g. “Report_hw1_312551120_陳柏憲.pdf”
- Name your .zip file as “mL_hw1_<yourStudentID>_<yourName>.zip”,
e.g. “ML_hw1_312551120_陳柏憲.zip”

(Uppercase or lowercase of the letters are fine)

Punishment Policy:

Except for what we mention in this file, please also refer to *project_policy.pdf*. (Important)

Good luck~

TAs